MODULE 25 · DOMAIN 4 - IMPLEMENT TEXT ANALYSIS SOLUTIONS

Extracting meaning from text

Named entity recognition, which Azure Language features are core or legacy, generative extraction into JSON, topics and summaries, and validating what a model extracts.

DOMAIN 4 - Text analysis WEIGHT 10-15% MODULE 25 of 39

admin@sthithapragna.com

A tool extracts what's **defined**. A model extracts what you **describe**.

admin@sthithapragna.com

Azure Language or a model with a schema?

AZURE LANGUAGE

Defined extraction

- Fixed categories: person, location, date
- Exact offsets and confidence scores
- Same input, same output
- Priced per text record, no prompt to write

GENERATIVE PROMPTING

Described extraction

- Any fields you can define in a schema
- Topics, summaries, domain-specific values
- Needs validation: values can be wrong
- Costs tokens, varies by prompt and model

ON THE EXAM

Both are Foundry capabilities. The skill expects you to choose between them for each field, not for the whole app.

Core and legacy capabilities

GROUP	FEATURES	GUIDANCE
Core	PII detection, language detection, prebuilt NER	Recommended for new development
Core	Custom NER, text analytics for health	Recommended for new development
Legacy	Key phrase extraction, sentiment and opinion mining	Existing implementations only
Legacy	Summarization, entity linking, custom text classification	Existing implementations only
Legacy	CLU, question answering, orchestration workflow	Existing implementations only

ON THE EXAM

Microsoft's own overview groups the features this way. Older material treats every row as a current recommendation.

Key phrases, summarization and entity linking are **legacy**, retiring by **2029**. Microsoft points new topic and summary work to **Foundry models**.

admin@sthithapragna.com

What prebuilt NER returns

- Categories such as **Person, Location, Organization, DateTime, Quantity, Email and URL**.
- Each entity has **text, category, subcategory, offset, length** and a **confidenceScore**.
- Newer model versions add **tags** and **metadata**, such as a number's actual value.
- **Synchronous** calls store nothing. **Asynchronous** results are kept for **24 hours**.
- If you don't specify a language, the service assumes **English**.

> ON THE EXAM
>
> Offsets make NER auditable: every entity points to exactly where it appears in the source.

An NER request and one entity in the response

```
{"kind": "EntityRecognition",
 "parameters": {"modelVersion": "latest"},
 "analysisInput": {"documents": [
   {"id": "1", "language": "en",
    "text": "Joe went to London on Saturday"}]}}

# One entity from the results:
{"text": "London", "category": "Location", "subcategory": "GPE",
 "offset": 12, "length": 6, "confidenceScore": 0.88}
```

ON THE EXAM

offset 12, length 6: characters 12 to 17 of the input spell 'London'. That's how you highlight it in the UI.

Offsets, encodings and emoji

- Offsets and lengths depend on the **string index type** used to count characters.
- **Emoji** and many non-Latin characters take more than one unit in some encodings.
- If your language counts characters differently from the service, highlights **drift**.
- Request the string index type that **matches your programming language**.
- In .NET, Microsoft recommends indexing with **StringInfo** for correct results.

> THE SIGNAL
>
> Entities highlighted in the wrong place, only in messages with emoji: an offset encoding mismatch.

Documents per request, and rates

FEATURE	DOCUMENTS PER REQUEST
NER, PII detection, entity linking	5
Key phrases, sentiment, opinion mining	10
Document summarization, health (web API)	25
Conversation summarization	1
Language detection	1,000

ON THE EXAM

A text record is 1,000 characters for billing. S0 allows 100 requests a second and 300 a minute, per feature.

Custom named entity recognition

- Train a model to extract **your own entity categories**, such as policy numbers or part codes.
- A **core** capability, recommended for new development.
- Needs **labelled examples** of your text, and time to train and evaluate.
- Suits **stable, high-volume** extraction where the categories rarely change.
- No labelled data, or categories still changing? A model with a schema starts faster.

ON THE EXAM

Custom NER keeps the tool's strengths, like offsets and consistency, for categories Microsoft didn't define.

Extracting JSON with a model

- Design the **schema first**: every field named, typed and described.
- Use **structured outputs** with strict set, as module 12 showed, so the shape is guaranteed.
- Allow **null** for fields that may be missing, so the model isn't forced to invent one.
- Ask for an **evidence quote** for each key value, copied from the source.
- Keep instructions strict: extract only what the text states, and don't infer.

> ON THE EXAM
>
> A required field with no null option is an invitation to fabricate. Missing is a valid answer.

Topics, summary and evidence in one schema

```
schema = {"type": "object", "additionalProperties": False,
  "required": ["topic", "summary", "order_id", "evidence"],
  "properties": {
    "topic": {"type": "string",
              "enum": ["delivery", "refund", "damage", "other"]},
    "summary": {"type": "string",
                "description": "Two sentences, customer's issue and request"},
    "order_id": {"type": ["string", "null"]},
    "evidence": {"type": "string",
                 "description": "Exact sentence from the ticket naming the issue"}}}
```

ON THE EXAM

The enum turns topic into classification. The null allows a missing order ID. Evidence makes the result checkable.

Extractive or abstractive?

EXTRACTIVE

Select sentences

- Picks key sentences from the source
- Wording is exactly the original
- Easy to verify, can read unevenly

ABSTRACTIVE

Write new sentences

- Produces new, concise wording
- Reads smoothly, fits any length
- Must be checked against the source

ON THE EXAM

With a model, ask for either style explicitly: 'quote the three key sentences' or 'summarise in your own words'.

Topics and summaries across long or many documents

- **Fixed topic lists** use an enum; **open topics** use an array of short strings.
- For long documents, **summarise chunks**, then summarise the summaries.
- Run chunk summaries **concurrently**, the fan-out and fan-in pattern from module 14.
- Pass the **same schema** to every chunk, so results merge cleanly.
- Watch quota: parallel chunks hit rate limits at the same moment.

> ON THE EXAM
>
> Open topics drift into synonyms, like 'refund' and 'money back'. A fixed enum keeps reporting consistent.

Validating what a model extracts

- A strict schema guarantees **shape**, not **truth**.
- Check values in code: dates parse, amounts add up, IDs match a known format.
- Confirm each **evidence quote** actually appears in the source text.
- Measure with **groundedness** and custom evaluators, as module 15 showed.
- Route failed checks and low-confidence results to a **person**.

> ON THE EXAM
>
> A simple string match on the evidence quote catches many fabricated extractions for free.

Azure Language as a tool

- The **Azure Language MCP server** exposes NER, PII detection, language detection and more.
- Available as a **remote** server in the Foundry Tool Catalog, or **local** and self-hosted.
- An agent can call NER for precise entities, then reason over them itself.
- **Containers** run some features on premises, for data that can't leave your network.
- Container options include **custom NER**, text analytics for health, and language detection.

> ON THE EXAM
>
> Agents combine both approaches naturally: a tool for exact entities, the model for everything described.

Requirement to approach

REQUIREMENT	APPROACH
People, places and dates, with exact positions	Prebuilt NER
Your own entity types, high volume, labelled data	Custom NER
Topics from a fixed list	Model, structured output with an enum
Readable summaries for a new app	Model, abstractive summary field
Domain fields like notice periods, no training data	Model with a schema, plus evidence
An existing key phrase or summarization app	Legacy Language features, plan migration
Extraction inside an agent	Language MCP server, or model reasoning

ON THE EXAM

Per field, ask: is it defined already, or do I have to describe it?

> A company is building a new support-ticket app. Each ticket needs a topic from a fixed list of eight, and a short summary for agents.
>
> An architect proposes Azure Language key phrase extraction for topics, and Azure Language summarization for summaries.

What should the team build instead?

A A Foundry model with a strict schema: an enum topic and a summary.

B The proposed Language key phrase extraction and summarization.

C Language entity linking to map each ticket to a Wikipedia topic.

D Prebuilt NER, treating Organization entities as each ticket's topic.

◆ PAUSE AND DECIDE

What should the team build instead?

A Model with enum topic and summary.
Classifies into the fixed list and writes the summary, in one call, on current guidance.

B Key phrases plus summarization.
Legacy features for existing apps. Key phrases also aren't a fixed topic list.

C Entity linking.
Legacy, retiring in 2028, and links entities to Wikipedia rather than your topics.

D NER organisations as topics.
Organisations mentioned aren't the ticket's topic.

THE SIGNAL

A new app, fixed topics and summaries: a model with a schema. Legacy Language features are for existing workloads.

> A chat app uses prebuilt NER to highlight people and places in customer messages. Most highlights are correct.
>
> But in messages that contain emoji, the highlights land a few characters away from the right words.

What should the team change?

A Raise the confidence threshold so that fewer entities are highlighted.

B Pin an older NER model version so the offsets are calculated as before.

C Request the string index type that matches how the app counts text.

D Strip every emoji from messages before displaying them to agents.

◆ PAUSE AND DECIDE

What should the team change?

A Higher confidence threshold.

Fewer highlights, but the remaining ones are still in the wrong place.

B Older model version.

The entities are right. The problem is how positions are counted.

C Matching string index type.

Offsets then count characters the same way the app does, emoji included.

D Strip emoji.

Changes what customers wrote to hide a counting mismatch.

THE SIGNAL

Correct entities, wrong positions, only with emoji: an offset encoding mismatch.

> A nightly job extracts entities from product reviews with prebuilt NER. It sends 50 reviews in each synchronous request.
>
> Every request fails, although the same code worked when a developer tested it with three reviews.

What should the team change?

A Upgrade the Language resource to a higher tier for larger requests.

B Send at most 5 documents per NER request, splitting the batch.

C Shorten each review to 1,000 characters before sending it.

D Switch to key phrase extraction, which accepts more documents.

◆ PAUSE AND DECIDE

What should the team change?

A. Higher tier.

Tiers change request rates, not how many documents one request can hold.

B. Five documents per request.

NER accepts up to 5 documents in each request. Fifty exceeds it.

C. Shorter reviews.

Document length isn't the issue. Three long reviews worked fine.

D. Key phrase extraction.

Returns phrases, not entities, and it's a legacy feature.

THE SIGNAL

'Works with three, fails with fifty' is a documents-per-request limit, and NER's is five.

> A legal team needs people, organisations and dates from contracts, with exact positions for audit. They also need domain fields, like the termination notice period.
>
> They have no labelled contracts, and need results this month.

What should the team build?

A. Prebuilt NER only, reading the notice period from the DateTime entities.

B. Custom NER, trained to recognise every standard and domain field.

C. Entity linking for parties, with the notice period left for manual review.

D. Prebuilt NER for standard entities, and a schema for domain fields.

◆ PAUSE AND DECIDE

What should the team build?

A Prebuilt NER only.

Finds '30 days' as a quantity, but not that it's the termination notice period.

B Custom NER for everything.

Needs labelled contracts, which don't exist, and training time.

C Entity linking, manual notice.

Legacy, retiring, and leaves the domain field unautomated.

D NER plus a model schema.

Exact offsets for standard entities; described fields, with evidence, for the rest.

THE TIEBREAKER

Defined entities with positions: NER. Described domain fields without training data: a model. Choose per field.

Extracting meaning in five lines

- **Tools extract what's defined; models extract what you describe.** Choose per field.
- **Core:** NER, custom NER, PII, language detection, health. **Legacy:** key phrases, sentiment, summarization.
- **NER returns offsets and confidence;** match the string index type. Five documents per request.
- **Model extraction:** strict schema, nulls allowed, enums for topics, evidence quotes.
- **Verify:** a schema guarantees shape, not truth.

> ON THE EXAM
>
> Next, module 26: sentiment, tone, safety and sensitive content, and customising outputs for domain tasks.